

Trabalho de Pesquisa – Web Services

Autor: [Anthero Franco Sprana]

1. Conceito de Web Service

Um Web Service pode ser definido como um sistema de software projetado para permitir a interação interoperável máquina-a-máquina através de uma rede, segundo definição do World Wide Web Consortium (W3C).

Ele opera como um componente que expõe funcionalidades ou dados para outros sistemas, independentemente da linguagem de programação ou plataforma usada. No desenvolvimento de sistemas, o uso de Web Services permite a reutilização de funcionalidades, a integração entre sistemas heterogêneos e a ampliação de ecossistemas de software, reduzindo custos e tempo de desenvolvimento.

2. Arquitetura e Funcionamento

2.1 Comunicação entre cliente e servidor

Na arquitetura típica de Web Services, há dois papéis principais: o cliente (ou consumidor) que faz a requisição, e o servidor (ou provedor) que processa essa requisição e retorna uma resposta. A comunicação ocorre via rede, geralmente sobre o protocolo HTTP, e utiliza formatos padronizados de dados (como XML ou JSON).

2.2 O papel dos protocolos HTTP, SOAP e REST

- **HTTP (Hypertext Transfer Protocol):** protocolo de transporte mais comum para requisições Web, servindo como base para muitos Web Services.
- **SOAP (Simple Object Access Protocol):** protocolo mais formal, baseado em XML, com suporte a especificações de segurança (WS-Security) e transações.
- **REST (Representational State Transfer):** estilo arquitetural mais leve, que utiliza os métodos HTTP (GET, POST, PUT, DELETE) e geralmente faz uso de JSON como formato de dados.

3. Tipos de Web Services

3.1 SOAP

- **Formato de dados:** XML.
- **Vantagens:** alta formalidade, padronização, suporte a segurança corporativa.
- **Desvantagens:** complexidade, maior sobrecarga.
- **Aplicações típicas:** sistemas enterprise, financeiros ou governamentais onde rigor e confiabilidade são críticos.

3.2 REST

- **Formato de dados:** frequentemente JSON (mas também pode usar XML).
- **Vantagens:** simplicidade, leveza, ampla adoção.
- **Desvantagens:** menor formalidade de contrato e padrões frente ao SOAP.
- **Aplicações típicas:** aplicações web/publicas, mobile, startups, integração rápida entre serviços.

3.3 GraphQL

- **Formato de dados:** JSON; permite que o cliente especifique exatamente quais dados deseja.
- **Vantagens:** evita over-fetching ou under-fetching de dados, consulta mais flexível.
- **Desvantagens:** curva de aprendizagem, necessidade de controle mais rigoroso de consultas.
- **Aplicações típicas:** APIs modernas com múltiplas fontes de dados ou clientes heterogêneos (web, mobile, IoT).

3.4 Comparativo

Estudos e artigos apontam que a escolha entre SOAP, REST e GraphQL depende fortemente do contexto e requisitos (ex.: desempenho, segurança, complexidade). [GeeksforGeeks](#)

4. Formatos de Dados Utilizados

4.1 JSON vs XML

- **JSON (JavaScript Object Notation):** mais leve, fácil de ler, muito usado em ambientes web e mobile.
- **XML (eXtensible Markup Language):** mais estruturado, com suporte a schemas, namespaces, mais formal — utilizado frequentemente em contextos corporativos e SOAP.

4.2 Quando escolher um ou outro?

Se o foco for integração rápida, desempenho e simplicidade, JSON tende a ser a melhor escolha. Se for necessário esquema rígido, compatibilidade com sistemas legados, ou funcionalidades como namespaces e validação forte, XML pode ser mais indicado.

5. Padrões e Boas Práticas

5.1 Documentação

Uma API ou Web Service bem documentado facilita adoção, manutenção e integração. Ferramentas como Swagger / OpenAPI Specification e coleções de requisição como Postman ajudam nesta tarefa.

5.2 Autenticação e Segurança

Métodos comuns incluem API Key, Token e JWT (JSON Web Token) para controle de acesso e autorização. Uso de HTTPS, limitação de requisições (rate limiting) e monitoramento são práticas essenciais.

5.3 Versionamento e Manutenção

Versionar endpoints (por exemplo: `/api/v1/usuarios`) evita que mudanças quebrem integrações existentes. Garantir compatibilidade e migrar clientes de forma controlada são medidas recomendadas.

6. Casos Reais e Aplicações

6.1 APIs Públicas Conhecidas

- ViaCEP API: consulta de endereços através do CEP no Brasil.
 - IBGE API: acesso a dados estatísticos e geográficos brasileiros.
 - OpenWeather API: serviço de previsão e dados meteorológicos.
- Esses serviços ilustram a forma como Web Services são disponibilizados e consumidos em larga escala.

6.2 Integração Empresarial

Empresas utilizam Web Services para conectar seus sistemas internos (ERP, CRM, logística), com parceiros externos, realizar automações, expor funcionalidades ou dados e construir ecossistemas de soluções colaborativas.

7. Considerações Finais

A adoção e correta implementação de Web Services permitiram o desenvolvimento de sistemas mais modulares, interoperáveis e escaláveis. A escolha adequada dos protocolos (SOAP, REST, GraphQL) e dos formatos de dados (JSON, XML) — além da aplicação de boas práticas como documentação, autenticação e versionamento — fazem diferença no sucesso das integrações. Para futuras pesquisas, analisar empiricamente o desempenho e evolução desses estilos pode iluminar decisões arquiteturais estratégicas.

Referências Principais

- (https://en.wikipedia.org/wiki/Web_service)
- (<https://opensoft.pt/2016/06/07/web-service/>)
- (<https://www.freecodecamp.org/news/rest-vs-graphql-apis/>)
- (<https://www.geeksforgeeks.org/rest-api-vs-graphql-vs-soap/>)
- (<https://www.techtarget.com/searcharchitecture/definition/Web-services>)
- (<https://www.solwey.com/posts/exploring-api-architecture-styles-rest-graphql-soap-and-beyond>)
- (https://www.w3schools.com/xml/dom_http.asp)
- (https://www.w3schools.com/nodejs/nodejs_rest_api.asp)